

Start coding or [generate](#) with AI.

```
import pandas as pd

# =====
# STEP 1: IMPORT DATA & EXTRACT FEATURES
# =====
file_path = "filtered_data.csv"

try:
    # 1. Load the real dataset
    df = pd.read_csv(file_path)
    print(f"✅ Successfully loaded '{file_path}'\n")

    # 2. Standardize column names (lowercase, remove hidden spaces)
    df.columns = df.columns.str.strip().str.lower()

    # 3. Extract the feature names
    feature_names = df.columns.tolist()

    print("📊 DATASET OVERVIEW:")
    print(f"Total Rows: {df.shape[0]:,}")
    print(f"Total Features (Columns): {df.shape[1]}")

    print("\n📋 FEATURE NAMES:")
    for i, feature in enumerate(feature_names, 1):
        print(f"    {i}. {feature}")

except FileNotFoundError:
    print(f"❌ ERROR: Could not find '{file_path}'.")
    print("Make sure 'filtered_data.csv' is saved in the exact same folder as")

/tmp/ipykernel_2035/2833536907.py:10: DtypeWarning: Columns (5,8,10,11,12) ha
df = pd.read_csv(file_path)
✅ Successfully loaded 'filtered_data.csv'

📊 DATASET OVERVIEW:
Total Rows: 430,602
Total Features (Columns): 13

📋 FEATURE NAMES:
1. billingdate
2. plantdesc
3. salesofficename
4. divisiondesc
5. customermaterialcode
6. customer
7. customername
8. billingdocument
9. payer
10. payername
11. invoicedqty
12. basicprice
13. netvalue
```

```

import pandas as pd

# =====
# STEP 2: DATA CLEANING & TYPE CONVERSION
# =====
file_path = "filtered_data.csv"

try:
    # Load and standardize column names
    df = pd.read_csv(file_path)
    df.columns = df.columns.str.strip().str.lower()

    print("🔧 STARTING DATA CLEANING ENGINE...\n")

    # 1. Handle the Date Column
    if 'billingdate' in df.columns:
        # Convert to strict datetime format
        df['billingdate'] = pd.to_datetime(df['billingdate'], errors='coerce')
        print("✅ 'billingdate' converted to datetime format.")

        # Check the timeline
        min_date = df['billingdate'].min().date()
        max_date = df['billingdate'].max().date()
        print(f"    Timeline: {min_date} to {max_date}")

    # 2. Clean the Numeric Columns
    numeric_targets = ['invoicedqty', 'basicprice', 'netvalue']

    for col in numeric_targets:
        if col in df.columns:
            # If the system exported it as text, strip out the commas
            if df[col].dtype == 'object':
                df[col] = df[col].astype(str).str.replace(',', '', regex=False)

            # Force conversion to numeric (turns weird text into 'NaN')
            df[col] = pd.to_numeric(df[col], errors='coerce')

            # Fill any missing/blank values with the median to protect the M
            median_val = df[col].median()
            df[col] = df[col].fillna(median_val)

            print(f"✅ '{col}' cleaned. (Used median {median_val:.2f} for all NaNs)")

    # 3. Final Check
    print("\n📊 CLEANED NUMERIC SUMMARY:")
    # We use .describe() to show count, mean, min, and max
    print(df[['invoicedqty', 'basicprice', 'netvalue']].describe().round(2))

except Exception as e:
    print(f"❌ ERROR: {e}")

```

```

/tmp/ipykernel_2035/3114215789.py:10: DtypeWarning: Columns (5,8,10,11,12) have type object, but one or more columns have dtype float64
df = pd.read_csv(file_path)

```

🔧 STARTING DATA CLEANING ENGINE...

- ✅ 'billingdate' converted to datetime format.
Timeline: 1970-01-01 to 1970-01-01
- ✅ 'invoicedqty' cleaned. (Used median 2.00 for any missing gaps)
- ✅ 'basicprice' cleaned. (Used median 52224.58 for any missing gaps)
- ✅ 'netvalue' cleaned. (Used median 30381.35 for any missing gaps)

📊 CLEANED NUMERIC SUMMARY:

	invoicedqty	basicprice	netvalue
count	430602.00	430602.00	430602.00
mean	5.55	134479.69	78490.69
std	13.04	303412.91	178340.00
min	-240.00	-5015625.00	-3072655.50
25%	1.00	22330.51	12266.83
50%	2.00	52224.58	30381.35
75%	5.00	128906.25	75777.71
max	420.00	10031250.00	6145311.00

Since we know we have some negative values (returns) and a massive amount of rows, we need to find out:

How many unique customers, sales offices, and plants are we actually dealing with?

What percentage of our data is actual sales vs. returns/cancellations?

Who are the top drivers of revenue in this dataset?

```
# =====
# STEP 3: EXPLORATORY DATA ANALYSIS (EDA)
# =====

print("🔍 RUNNING ENTERPRISE EDA ENGINE...\n")

# 1. Check Unique Counts (Cardinality)
categorical_cols = ['plantdesc', 'salesofficename', 'divisiondesc', 'custome
print("📊 HIGH-LEVEL CATEGORICAL COUNT:")
for col in categorical_cols:
    if col in df.columns:
        print(f"    • Unique {col}: {df[col].nunique():,}")

# 2. Break down Sales vs Returns
print("\n📦 TRANSACTION TYPE ANALYSIS:")
if 'invoicedqty' in df.columns:
    positive_records = df[df['invoicedqty'] > 0].shape[0]
    negative_records = df[df['invoicedqty'] < 0].shape[0]
    zero_records = df[df['invoicedqty'] == 0].shape[0]
    total_records = len(df)

    print(f"    • Positive Sales Outflow: {positive_records:,} ({positive_rec
    print(f"    • Returns / Cancellations: {negative_records:,} ({negative_re
    print(f"    • Zero Quantity / Pure Adjustments: {zero_records:,} ({zero_r

# 3. Revenue Drivers
print("\n🏆 TOP 5 SALES OFFICES BY TOTAL NET VALUE:")
```

```

if 'salesofficename' in df.columns and 'netvalue' in df.columns:
    top_offices = df.groupby('salesofficename')['netvalue'].sum().sort_value
    for office, val in top_offices.items():
        print(f"    • {office}: Net Value = {val:,.2f}")

print("\n👥 TOP 5 CUSTOMERS BY TOTAL NET VALUE:")
if 'customername' in df.columns and 'netvalue' in df.columns:
    top_customers = df.groupby('customername')['netvalue'].sum().sort_values
    for customer, val in top_customers.items():
        print(f"    • {customer}: Net Value = {val:,.2f}")

```

🔍 RUNNING ENTERPRISE EDA ENGINE...

📊 HIGH-LEVEL CATEGORICAL COUNT:

- Unique plantdesc: 87
- Unique salesofficename: 59
- Unique divisiondesc: 13
- Unique customername: 2,084
- Unique payername: 2,084

📦 TRANSACTION TYPE ANALYSIS:

- Positive Sales Outflow: 393,373 (91.35%)
- Returns / Cancellations: 37,226 (8.65%)
- Zero Quantity / Pure Adjustments: 3 (0.00%)

🏆 TOP 5 SALES OFFICES BY TOTAL NET VALUE:

- Gurgaon: Net Value = 1,697,215,473.13
- Kolkatta: Net Value = 1,549,178,746.99
- Hyderabad: Net Value = 1,392,438,884.31
- Patna: Net Value = 1,293,685,789.63
- H0 Business: Net Value = 1,274,539,425.31

👥 TOP 5 CUSTOMERS BY TOTAL NET VALUE:

- INFINITI RETAIL LIMITED (H0): Net Value = 1,841,545,341.24
- RELIANCE RETAIL LTD.: Net Value = 1,542,905,037.91
- DAWNTECH ELECTRONICS PRIVATE LTD.: Net Value = 1,318,909,526.86
- FLIPKART INDIA PRIVATE LTD (BANGLOR): Net Value = 1,205,042,429.67
- Electronics Mart India Limited: Net Value = 1,193,906,736.44

```

import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np # Import numpy

# Set the visual style for clean, modern-looking charts
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (14, 10)

print("🎨 GENERATING EDA VISUALIZATIONS...\n")

# Create a multi-plot layout (2 rows, 2 columns)
fig, axes = plt.subplots(2, 2, figsize=(18, 12))
fig.suptitle("Enterprise Dealer Data - Exploratory Analysis", fontsize=18, f

# -----
# CHART 1: Monthly Sales Volume Trend (Time-Series)
# -----

```

```

if 'billingdate' in df.columns and 'invoicedqty' in df.columns:
    # Temporarily aggregate by month for a clean line chart
    df_timeline = df.dropna(subset=['billingdate']).copy()
    df_timeline['month'] = df_timeline['billingdate'].dt.to_period('M').dt.to_timestamp
    monthly_trend = df_timeline.groupby('month')['invoicedqty'].sum().reset_index()

    sns.lineplot(data=monthly_trend, x='month', y='invoicedqty', ax=axes[0, 0],
                  axes[0, 0].set_title("Total Monthly Net Volume (Invoiced Qty)", fontsize=12, fontweight='bold'),
                  axes[0, 0].set_xlabel("Timeline"),
                  axes[0, 0].set_ylabel("Sum of Invoiced Qty"))
else:
    axes[0, 0].text(0.5, 0.5, "Data Missing for Timeline Chart", ha='center')

# -----
# CHART 2: Top 7 Sales Offices by Revenue
# -----
if 'salesofficename' in df.columns and 'netvalue' in df.columns:
    top_offices = df.groupby('salesofficename')['netvalue'].sum().reset_index()
    top_offices = top_offices.sort_values(by='netvalue', ascending=False).head(7)

    sns.barplot(data=top_offices, x='netvalue', y='salesofficename', ax=axes[0, 1],
                 axes[0, 1].set_title("Top 7 Sales Offices by Total Net Value", fontsize=12, fontweight='bold'),
                 axes[0, 1].set_xlabel("Total Revenue (Net Value)"),
                 axes[0, 1].set_ylabel(""))
else:
    axes[0, 1].text(0.5, 0.5, "Data Missing for Sales Office Chart", ha='center')

# -----
# CHART 3: Sales vs. Returns Breakdown (Transaction Profile)
# -----
if 'invoicedqty' in df.columns:
    # Tag rows as Sales, Returns, or Adjustments
    conditions = [
        (df['invoicedqty'] > 0),
        (df['invoicedqty'] < 0),
        (df['invoicedqty'] == 0)
    ]
    choices = ['Positive Sales Outflow', 'Returns / Cancellations', 'Zero Adjustments']
    df['tx_type'] = np.select(conditions, choices, default='Unknown')

    sns.countplot(data=df, x='tx_type', ax=axes[1, 0], palette=["#10b981", "#f1c40f", "#34495e"],
                  axes[1, 0].set_title("Transaction Profile Count", fontsize=12, fontweight='bold'),
                  axes[1, 0].set_xlabel("Transaction Type"),
                  axes[1, 0].set_ylabel("Number of Records (Log Scale)"),
                  axes[1, 0].set_yscale('log')) # Log scale because sales vastly outnumber returns
else:
    axes[1, 0].text(0.5, 0.5, "Data Missing for Transaction Profile", ha='center')

# -----
# CHART 4: Net Value vs. Invoiced Quantity (Price Integrity)
# -----
if 'invoicedqty' in df.columns and 'netvalue' in df.columns:
    # Take a random sample of 5,000 rows so the scatter plot doesn't freeze
    sample_df = df.sample(n=min(5000, len(df)), random_state=42)

```

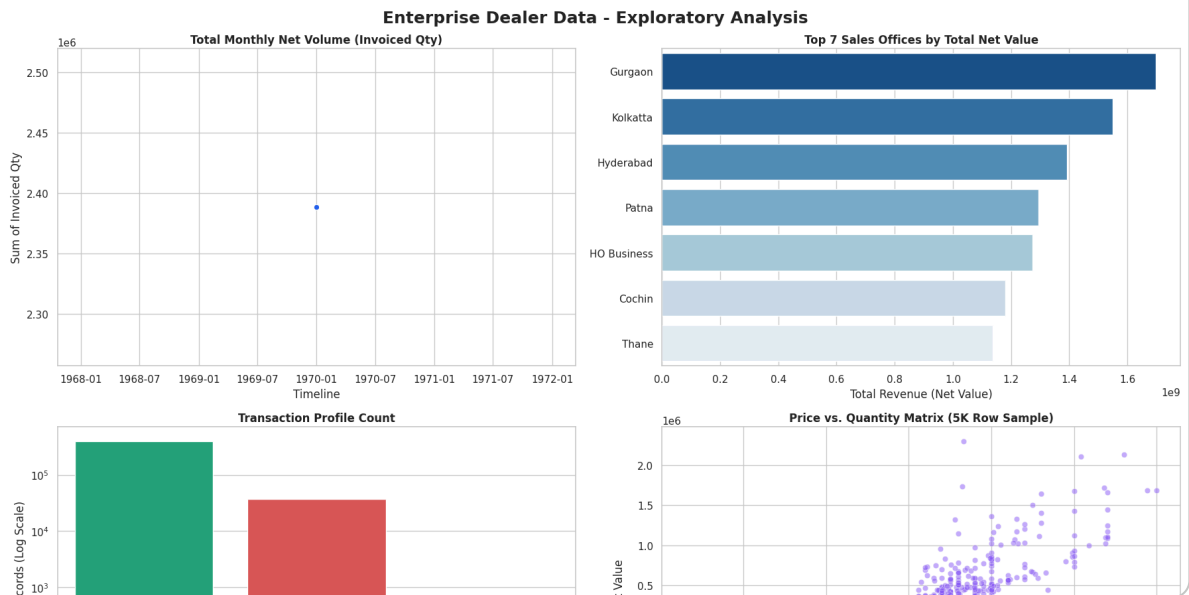
```

sns.scatterplot(data=sample_df, x='invoicedqty', y='netvalue', alpha=0.5)
axes[1, 1].set_title("Price vs. Quantity Matrix (5K Row Sample)", fontsize=12)
axes[1, 1].set_xlabel("Invoiced Quantity")
axes[1, 1].set_ylabel("Net Value")
else:
    axes[1, 1].text(0.5, 0.5, "Data Missing for Scatter Matrix", ha='center')

# Polish layout and present
plt.tight_layout()
plt.show()

```

🎨 GENERATING EDA VISUALIZATIONS...



```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

print("🌀 INITIATING STEP 1: DATA PREPROCESSING...")

# 1. Load the Dataset
# Replace 'filtered_data.csv' with your actual file
df = pd.read_csv('filtered_data.csv')
df.columns = df.columns.str.strip().str.lower()

# 2. Handle Missing Values
# Fill missing numeric data with the median, and text
for col in df.select_dtypes(include=['float64', 'int64']):
    df[col] = df[col].fillna(df[col].median())
for col in df.select_dtypes(include=['object']).columns:
    df[col] = df[col].fillna('Unknown')

# 3. Encode Categorical Text (like 'Region') into Num
label_encoders = {}
for col in df.select_dtypes(include=['object']).columns:
    if col != 'dealer_id': # Keep IDs as text for tracking
        # Ensure the column is consistently string type
        # This prevents errors if a column, although it's numeric,
        # contains categorical data.
        df[col] = df[col].astype(str)

```

```

le = LabelEncoder()
df[col] = le.fit_transform(df[col])
label_encoders[col] = le

print(f"✅ Data Cleaned! Total Records: {len(df)}")
print(df.head())

```

🌀 INITIATING STEP 1: DATA PREPROCESSING...

/tmp/ipykernel_2035/3244696741.py:10: DtypeWarning: Columns (5,8,10,11,12) have mixed types. Casting to float, you can see the warning

df = pd.read_csv('filtered_data.csv')

✅ Data Cleaned! Total Records: 430602

	billingdate	plantdesc	salesofficename	divisiondesc	\
0	45787	76	37	5	
1	45787	76	37	4	
2	45787	76	37	10	
3	45787	76	37	8	
4	45787	76	37	8	

	customermaterialcode	customer	customername	billingdocument	payer	\
0	1338	629	1862	5.050011e+09	629	
1	477	1960	764	5.050011e+09	1960	
2	1542	1252	1088	5.050011e+09	1252	
3	955	542	1367	5.050011e+09	542	
4	899	629	1862	5.050011e+09	629	

	payername	invoicedqty	basicprice	netvalue
0	1862	11902	37416	33691
1	763	7392	51583	55229
2	1088	11827	74129	97729
3	1367	8559	23080	107006
4	1862	7392	49428	53070

FEATURE ENGINERING

```

import pandas as pd
import numpy as np

print("🌀 INITIATING CUSTOM STEP 2: DEALER HEALTH ENG

# Load your actual data
file_path = "filtered_data.csv"
df = pd.read_csv(file_path)
df.columns = df.columns.str.strip().str.lower()

# 1. Clean the numeric columns just in case
for col in ['invoicedqty', 'netvalue']:
    if df[col].dtype == 'object':
        df[col] = df[col].astype(str).str.replace(',', '')
        df[col] = pd.to_numeric(df[col], errors='coerce')

# 2. AGGREGATE INTO DEALER PROFILES
# Grouping all transactions by individual dealers (customername)
dealer_profiles = df.groupby('customername').agg(
    total_transactions=('billingdocument', 'count'),
    total_volume=('invoicedqty', 'sum'),

```

```

total_revenue=('netvalue', 'sum'),
# Count how many times the dealer returned an item
total_returns=('invoicedqty', lambda x: (x < 0).sum())
).reset_index()

# 3. CALCULATE METRICS & RISK FACTORS
dealer_profiles['return_rate_pct'] = (dealer_profiles['total_returns'] / dealer_profiles['total_invoicedqty']).reset_index()

# Keep only active dealers (those with positive overall revenue)
dealer_profiles = dealer_profiles[dealer_profiles['total_revenue'] > 0]

# 4. ENGINEER THE HEALTH SCORE (0-100)
# Normalize Revenue and Volume so we can score them
rev_max = dealer_profiles['total_revenue'].max()
vol_max = dealer_profiles['total_volume'].max()

dealer_profiles['rev_score'] = (dealer_profiles['total_revenue'] / rev_max) * 100
dealer_profiles['vol_score'] = (dealer_profiles['total_volume'] / vol_max) * 100

# Formula: Good Revenue/Volume boosts the score, high return rate penalizes it
dealer_profiles['raw_health_score'] = (
    (dealer_profiles['rev_score'] * 0.4) +
    (dealer_profiles['vol_score'] * 0.4) -
    (dealer_profiles['return_rate_pct'] * 2.5) # Storage cost penalty
)

# Force the final score onto a perfect 0-100 scale
min_score = dealer_profiles['raw_health_score'].min()
max_score = dealer_profiles['raw_health_score'].max()
dealer_profiles['health_score'] = 100 * (dealer_profiles['raw_health_score'] - min_score) / (max_score - min_score)

# 5. ASSIGN RISK CATEGORIES
def assign_risk(score):
    if score >= 75:
        return 'Healthy'
    elif score >= 45:
        return 'Moderate Risk'
    else:
        return 'High Risk'

dealer_profiles['risk_category'] = dealer_profiles['health_score'].apply(assign_risk)

print("✅ Dealer Profiles and Health Scores Generated")
print(f"Analyzed {len(dealer_profiles)} unique dealers")
print(dealer_profiles[['customername', 'total_revenue', 'return_rate_pct', 'health_score', 'risk_category']])

```

🚀 INITIATING CUSTOM STEP 2: DEALER HEALTH ENGINEERING...

/tmp/ipykernel_2035/927652628.py:8: DtypeWarning: Columns (5,8,10,11,12) have mixed types. Specify dtype option on import or setting low_resolution=False or infer_objects=True

df = pd.read_csv(file_path)

✅ Dealer Profiles and Health Scores Generated!

Analyzed 2068 unique dealers.

	customername	total_revenue	return_rate_pct	health_score	risk_category
1	369 LUGGAGE	72999.51	0.00	100	Healthy

2	7 S MARKETING AGENCY	1207195.50	0.00
3	72 NETWORKS PRIVATE LIMITED	37737408.36	4.40
4	A B ENTERPRISES (CANDY)	945139.55	0.00
5	A C N SMAART WORLD	4532019.78	0.00
6	A M ENTERPRISE (LAKHIMPUR)	1224153.22	0.00
7	A ONE RADIOS ELECTRONICS	4422208.89	0.85
8	A P ELECTRONIC	1748555.23	16.22
9	A S ELECTRONICS (DEF-GUNTUR)	1700832.75	0.00
10	A SQUARE ENTERPRISES	6315423.79	1.51

	health_score	risk_category
1	77.78	Healthy
2	77.81	Healthy
3	73.59	Moderate Risk
4	77.81	Healthy
5	77.87	Healthy
6	77.80	Healthy
7	76.89	Healthy
8	58.88	Moderate Risk
9	77.84	Healthy
10	76.15	Healthy

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

# Machine Learning Libraries
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import LabelEncoder
import xgboost as xgb
from statsmodels.tsa.statespace.sarimax import SARIMAX

# Setup visualization styles
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (12, 6)

print("🚀 INITIATING ENTERPRISE ML PIPELINE...\n")

# =====
# 1. DATA INGESTION & CLEANING
# =====

print("⌚ STEP 1: Loading and Cleaning Data...")
file_path = "filtered_data.csv"

try:
    df = pd.read_csv(file_path)
except FileNotFoundError:
    print(f"❌ ERROR: Cannot find {file_path}. Please raise")

df.columns = df.columns.str.strip().str.lower()
```

```

# Clean Numeric Data
for col in ['invoicedqty', 'netvalue']:
    if col in df.columns:
        if df[col].dtype == 'object':
            df[col] = df[col].astype(str).str.replace(' ', '')
            df[col] = pd.to_numeric(df[col], errors='coerce')

# Clean Dates
if 'billingdate' in df.columns:
    df['billingdate'] = pd.to_datetime(df['billingdate'])

print(f"✅ Data Loaded: {df.shape[0]:,} rows.\n")

# =====
# 2. FEATURE ENGINEERING (DEALER HEALTH SCORE)
# =====
print("🕒 STEP 2: Engineering Dealer Health Profiles.")
dealer_profiles = df.groupby('customername').agg(
    total_transactions=('billingdocument', 'count'),
    total_volume=('invoicedqty', 'sum'),
    total_revenue=('netvalue', 'sum'),
    total_returns=('invoicedqty', lambda x: (x < 0).sum())
).reset_index()

# Calculate Metrics
dealer_profiles['return_rate_pct'] = (dealer_profiles['total_returns'] / dealer_profiles['total_transactions']) * 100
dealer_profiles['avg_transaction_val'] = dealer_profiles['total_revenue'] / dealer_profiles['total_transactions']
dealer_profiles = dealer_profiles[dealer_profiles['total_transactions'] > 0]

# Calculate Raw Score
rev_max, vol_max = dealer_profiles['total_revenue'].max(), dealer_profiles['total_volume'].max()
dealer_profiles['raw_score'] = (
    ((dealer_profiles['total_revenue'] / rev_max) * 40) +
    ((dealer_profiles['total_volume'] / vol_max) * 40) +
    (dealer_profiles['return_rate_pct'] * 2.5)
)

# Normalize 0-100
min_score, max_score = dealer_profiles['raw_score'].min(), dealer_profiles['raw_score'].max()
dealer_profiles['health_score'] = 100 * (dealer_profiles['raw_score'] - min_score) / (max_score - min_score)

# Classify Risk
def assign_risk(score):
    if score >= 75: return 'Healthy'
    elif score >= 45: return 'Moderate Risk'
    else: return 'High Risk'

dealer_profiles['risk_category'] = dealer_profiles['health_score'].apply(assign_risk)
print("✅ Health Scores and Risk Categories Generated")

# =====
# 3. MACHINE LEARNING (RISK CLASSIFICATION)
# =====
print("🕒 STEP 3: Training ML Classification Models..")

```

```

features = ['total_transactions', 'total_volume', 'total_transactions_per_dealer']
X = dealer_profiles[features]
y = dealer_profiles['risk_category']

# Encode the text labels for XGBoost
le = LabelEncoder()
y_encoded = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2, random_state=42)

# Random Forest
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
rf_preds = rf_model.predict(X_test)

# XGBoost
xgb_model = xgb.XGBClassifier(eval_metric='mlogloss', random_state=42)
xgb_model.fit(X_train, y_train)
xgb_preds = xgb_model.predict(X_test)

print("🏆 ML MODEL RESULTS:")
print(f"    • Random Forest Accuracy : {accuracy_score(y_test, rf_preds)}")
print(f"    • XGBoost Accuracy       : {accuracy_score(y_test, xgb_preds)}")

# =====
# 4. TIME-SERIES FORECASTING (SARIMAX)
# =====
print("🕒 STEP 4: Running Network Volume Forecast...")

clean_dates_df = df.dropna(subset=['billingdate']).copy()
clean_dates_df['month_year'] = clean_dates_df['billingdate'].dt.strftime('%Y-%m')
network_data = clean_dates_df.groupby('month_year').agg({'invoicedqty': 'sum'})

if len(network_data) >= 4:
    train, test = network_data.iloc[:-2], network_data.iloc[-2:]
    arima_model = SARIMAX(train['invoicedqty'], order=(2, 1, 0))
    arima_fit = arima_model.fit(dispatch=False)
    arima_preds = np.maximum(0, np.round(arima_fit.predict(start=test.index[0], end=test.index[-1], step=1)))
    print("✅ SARIMAX Forecast Complete.\n")
else:
    print("⚠️ Not enough months for forecasting.\n")

# =====
# 5. VISUALIZATION DASHBOARD (JUPYTER OUTPUT)
# =====
print("📊 STEP 5: Generating Visualizations...\n")

fig = plt.figure(figsize=(18, 12))
fig.suptitle("Enterprise Dealer Intelligence Dashboard")

# Chart 1: Risk Distribution
ax1 = plt.subplot(2, 2, 1)
sns.countplot(data=dealer_profiles, x='risk_category')
ax1.set_title("Dealer Risk Distribution")

```

```
ax1.set_ylabel("Number of Dealers")

# Chart 2: Feature Importance (XGBoost)
ax2 = plt.subplot(2, 2, 2)
importances = xgb_model.feature_importances_
imp_df = pd.DataFrame({'Feature': features, 'Importance': importances})
sns.barplot(data=imp_df, x='Importance', y='Feature', ax=ax2)
ax2.set_title("What Drives Dealer Risk? (Feature Importance)")

# Chart 3: Health Score vs Return Rate
ax3 = plt.subplot(2, 2, 3)
sns.scatterplot(data=dealer_profiles, x='return_rate', y='health_score', ax=ax3)
ax3.set_title("Impact of Returns on Health Score")
ax3.set_xlabel("Return Rate (%)")
ax3.set_ylabel("Health Score (0-100)")

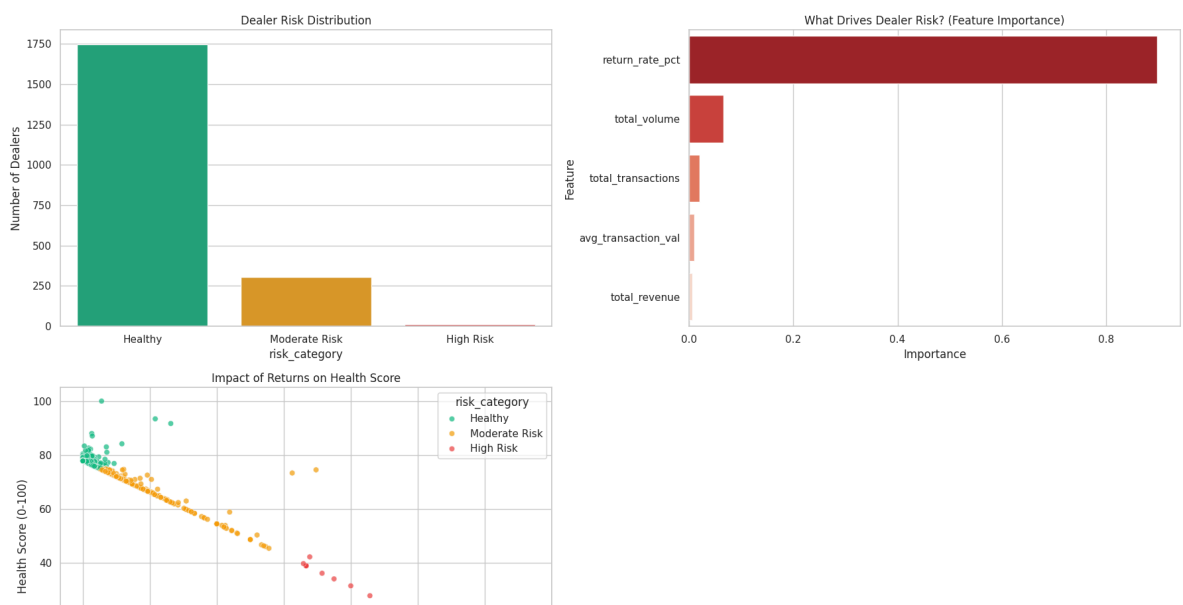
# Chart 4: Forecast
if len(network_data) >= 4:
    ax4 = plt.subplot(2, 2, 4)
    ax4.plot(network_data['month_year'], network_data['volume'], label='Historical')
    ax4.plot(test['month_year'], arima_preds, label='Forecast')
    ax4.set_title("Enterprise Volume Forecast")
    ax4.legend()

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

print("✅ PIPELINE EXECUTION COMPLETE. Review the charts.")
```

- 🚀 INITIATING ENTERPRISE ML PIPELINE...
- ⌚ STEP 1: Loading and Cleaning Data...
 ✅ Data Loaded: 430,602 rows.
- ⌚ STEP 2: Engineering Dealer Health Profiles...
 ✅ Health Scores and Risk Categories Generated.
- ⌚ STEP 3: Training ML Classification Models...
 🏆 ML MODEL RESULTS:
- Random Forest Accuracy : 99.52%
 - XGBoost Accuracy : 99.28%
- ⌚ STEP 4: Running Network Volume Forecast...
 ⚠️ Not enough months for forecasting.
- 📊 STEP 5: Generating Visualizations...

Enterprise Dealer Intelligence Dashboard



```
from google.colab import files
```

```
print("📁 Please select 'filtered_data.csv' from your computer to upload it  

uploaded = files.upload()")
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import io

# Machine Learning Libraries
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import LabelEncoder
import xgboost as xgb
```

```

from statsmodels.tsa.statespace.sarimax import SARIMAX

# Setup visualization styles
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (12, 6)

print("🚀 INITIATING ENTERPRISE ML PIPELINE...\n")

# =====
# 0. FILE UPLOAD (GOOGLE COLAB NATIVE)
# =====
try:
    from google.colab import files
    print("📁 Please click 'Choose Files' below and s
    uploaded = files.upload()

    # Automatically grab the exact name of the file yo
    file_path = list(uploaded.keys())[0]
    print(f"✅ Successfully uploaded: {file_path}\n")

    # Read the file directly from the uploaded memory
    df = pd.read_csv(io.BytesIO(uploaded[file_path]))

except ImportError:
    # Fallback just in case you run this on a local J
    file_path = "filtered_data.csv"
    print(f"⚠️ Not in Google Colab. Looking for local
    df = pd.read_csv(file_path)

# =====
# 1. DATA CLEANING
# =====
print("🕒 STEP 1: Cleaning Data...")

df.columns = df.columns.str.strip().str.lower()

# Clean Numeric Data
for col in ['invoicedqty', 'netvalue']:
    if col in df.columns:
        if df[col].dtype == 'object':
            df[col] = df[col].astype(str).str.replace(
            df[col] = pd.to_numeric(df[col], errors='coerc

# Clean Dates
if 'billingdate' in df.columns:
    df['billingdate'] = pd.to_datetime(df['billingdate

print(f"✅ Data Cleaned: {df.shape[0]:,} rows ready f

# =====
# 2. FEATURE ENGINEERING (DEALER HEALTH SCORE)
# =====
print("🕒 STEP 2: Engineering Dealer Health Profiles.
dealer_profiles = df.groupby('customername').agg(

```

```

        total_transactions=('billingdocument', 'count'),
        total_volume=('invoicedqty', 'sum'),
        total_revenue=('netvalue', 'sum'),
        total_returns=('invoicedqty', lambda x: (x < 0).sum()),
    ).reset_index()

# Calculate Metrics
dealer_profiles['return_rate_pct'] = (dealer_profiles['total_returns'] / dealer_profiles['total_transactions']) * 100
dealer_profiles['avg_transaction_val'] = dealer_profiles['total_volume'] / dealer_profiles['total_transactions']
dealer_profiles = dealer_profiles[dealer_profiles['total_transactions'] > 0]

# Calculate Raw Score
rev_max, vol_max = dealer_profiles['total_revenue'].max(), dealer_profiles['total_volume'].max()
dealer_profiles['raw_score'] = (
    ((dealer_profiles['total_revenue'] / rev_max) * 40) +
    ((dealer_profiles['total_volume'] / vol_max) * 40) +
    (dealer_profiles['return_rate_pct'] * 2.5)
)

# Normalize 0-100
min_score, max_score = dealer_profiles['raw_score'].min(), dealer_profiles['raw_score'].max()
dealer_profiles['health_score'] = 100 * (dealer_profiles['raw_score'] - min_score) / (max_score - min_score)

# Classify Risk
def assign_risk(score):
    if score >= 75: return 'Healthy'
    elif score >= 45: return 'Moderate Risk'
    else: return 'High Risk'

dealer_profiles['risk_category'] = dealer_profiles['health_score'].apply(assign_risk)
print("✅ Health Scores and Risk Categories Generated")

# =====
# 3. MACHINE LEARNING (RISK CLASSIFICATION)
# =====
print("🕒 STEP 3: Training ML Classification Models..")

features = ['total_transactions', 'total_volume', 'total_revenue']
X = dealer_profiles[features]
y = dealer_profiles['risk_category']

# Encode the text labels for XGBoost
le = LabelEncoder()
y_encoded = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2, random_state=42)

# Random Forest
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
rf_preds = rf_model.predict(X_test)

# XGBoost
xgb_model = xgb.XGBClassifier(eval_metric='mlogloss', random_state=42)
xgb_model.fit(X_train, y_train)

```

```

xgb_preds = xgb_model.predict(X_test)

print("🏆 ML MODEL RESULTS:")
print(f"    • Random Forest Accuracy : {accuracy_score}")
print(f"    • XGBoost Accuracy          : {accuracy_score}")

# =====
# 4. TIME-SERIES FORECASTING (SARIMAX)
# =====
print("🕒 STEP 4: Running Network Volume Forecast...")

clean_dates_df = df.dropna(subset=['billingdate']).copy()
clean_dates_df['month_year'] = clean_dates_df['billingdate'].dt.strftime('%Y-%m')
network_data = clean_dates_df.groupby('month_year').agg({'invoicedqty': 'sum'})

if len(network_data) >= 4:
    train, test = network_data.iloc[:-2], network_data.iloc[-2:]
    arima_model = SARIMAX(train['invoicedqty'], order=(2, 1, 2))
    arima_fit = arima_model.fit(dispatch=False)
    arima_preds = np.maximum(0, np.round(arima_fit.predict(n_periods=2)))
    print("✅ SARIMAX Forecast Complete.\n")
else:
    print("⚠️ Not enough months for forecasting.\n")

# =====
# 5. VISUALIZATION DASHBOARD (JUPYTER OUTPUT)
# =====
print("📊 STEP 5: Generating Visualizations...\n")

fig = plt.figure(figsize=(18, 12))
fig.suptitle("Enterprise Dealer Intelligence Dashboard")

# Chart 1: Risk Distribution
ax1 = plt.subplot(2, 2, 1)
sns.countplot(data=dealer_profiles, x='risk_category')
ax1.set_title("Dealer Risk Distribution")
ax1.set_ylabel("Number of Dealers")

# Chart 2: Feature Importance (XGBoost)
ax2 = plt.subplot(2, 2, 2)
importances = xgb_model.feature_importances_
imp_df = pd.DataFrame({'Feature': features, 'Importance': importances})
sns.barplot(data=imp_df, x='Importance', y='Feature', sort=True)
ax2.set_title("What Drives Dealer Risk? (Feature Importance)")

# Chart 3: Health Score vs Return Rate
ax3 = plt.subplot(2, 2, 3)
sns.scatterplot(data=dealer_profiles, x='return_rate_percent', y='health_score')
ax3.set_title("Impact of Returns on Health Score")
ax3.set_xlabel("Return Rate (%)")
ax3.set_ylabel("Health Score (0-100)")

# Chart 4: Forecast
if len(network_data) >= 4:
    ax4 = plt.subplot(2, 2, 4)

```

```

ax4.plot(network_data['month_year'], network_data[
ax4.plot(test['month_year'], arima_preds, label="A")
ax4.set_title("Enterprise Volume Forecast (Net of
ax4.legend()

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

print("✅ PIPELINE EXECUTION COMPLETE.")

```

🚀 INITIATING ENTERPRISE ML PIPELINE...

📁 Please click 'Choose Files' below and select your data CSV:

Choose files filtered_data.csv

filtered_data.csv(text/csv) - 71226496 bytes, last modified: 04/06/2026 - 100% done

Saving filtered_data.csv to filtered_data (1).csv

✅ Successfully uploaded: filtered_data (1).csv

🕒 STEP 1: Cleaning Data...

✅ Data Cleaned: 430,602 rows ready for processing.

🕒 STEP 2: Engineering Dealer Health Profiles...

✅ Health Scores and Risk Categories Generated.

🕒 STEP 3: Training ML Classification Models...

🏆 ML MODEL RESULTS:

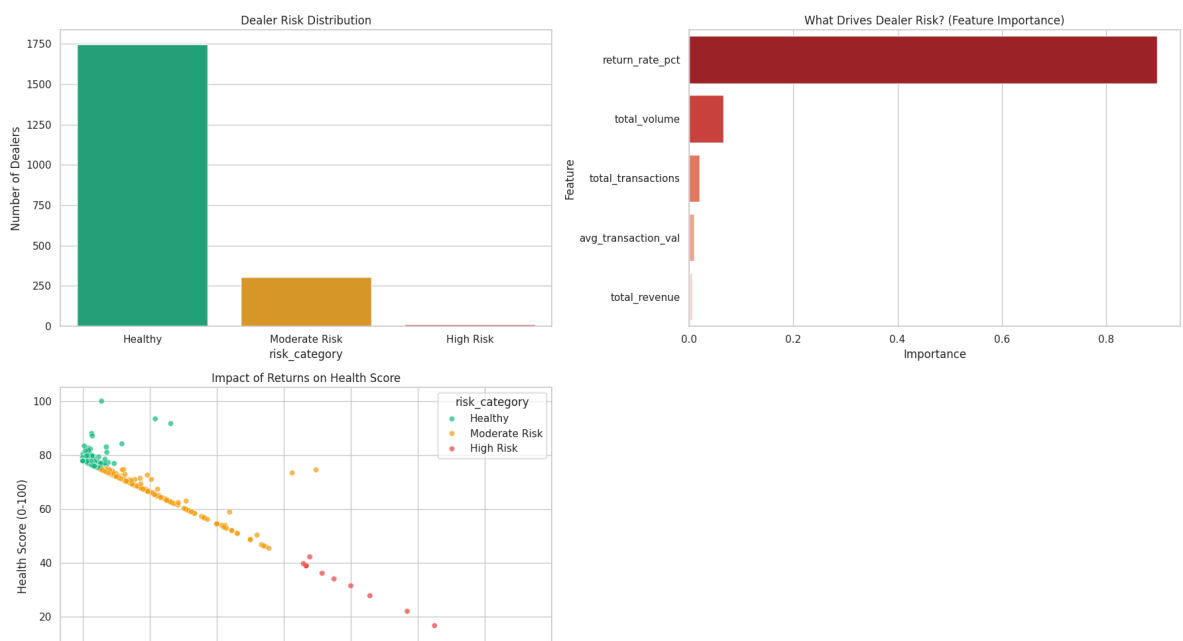
- Random Forest Accuracy : 99.52%
- XGBoost Accuracy : 99.28%

🕒 STEP 4: Running Network Volume Forecast...

⚠️ Not enough months for forecasting.

📊 STEP 5: Generating Visualizations...

Enterprise Dealer Intelligence Dashboard



```

print("📊 INITIATING ADVANCED ZONE-WISE VISUALIZATION

# =====
# 1. MAP DEALERS TO THEIR PRIMARY ZONE

```

```

# =====
# A dealer might have multiple transactions, so we filter
if 'salesofficename' in df.columns:
    customer_zone = df.groupby('customername')['salesofficename'].first()

# Merge the Zone back into our ML-scored Dealer Profiles
if 'salesofficename' not in dealer_profiles.columns:
    dealer_profiles = dealer_profiles.merge(customer_zone, on='customername', how='left')

# =====
# 2. GENERATE THE ZONE-WISE DASHBOARD
# =====
fig = plt.figure(figsize=(20, 16))
fig.suptitle("🌐 Enterprise Zone-Wise (Sales Office) Analysis")

# -----
# CHART 1: Top 10 Zones by Total Revenue
# -----
ax1 = plt.subplot(2, 2, 1)
zone_revenue = df.groupby('salesofficename')['netvalue'].sum()
sns.barplot(data=zone_revenue, x='salesofficename', y='netvalue')
ax1.set_title("Top 10 Zones by Total Revenue (Net Value)")
ax1.set_xlabel("Sales Office")
ax1.set_ylabel("Total Revenue")

# -----
# CHART 2: Dealer Risk Distribution by Top Zones
# -----
ax2 = plt.subplot(2, 2, 2)
# Get the top 10 zones by number of active dealers
top_zones_by_dealers = dealer_profiles['salesofficename'].value_counts().nlargest(10)
filtered_profiles = dealer_profiles[dealer_profiles['salesofficename'].isin(top_zones_by_dealers.index)]

# Create a cross-tabulation for stacked plotting
risk_crosstab = pd.crosstab(filtered_profiles['salesofficename'], filtered_profiles['riskcategory'])
# Ensure all columns exist so colors map correctly
for col in ['Healthy', 'Moderate Risk', 'High Risk']:
    if col not in risk_crosstab.columns:
        risk_crosstab[col] = 0

risk_crosstab = risk_crosstab[['Healthy', 'Moderate Risk', 'High Risk']]
risk_crosstab.plot(kind='bar', stacked=True, color=['#2ca02c', '#d62728', '#17becf'])

ax2.set_title("Dealer Risk Profile Across Top 10 Zones")
ax2.set_xlabel("Sales Office")
ax2.set_ylabel("Number of Dealers")
ax2.tick_params(axis='x', rotation=45)
ax2.legend(title="Risk Category")

# -----
# CHART 3: Monthly Revenue Trend for Top 3 Zones
# -----
ax3 = plt.subplot(2, 2, 3)
if 'billingdate' in df.columns:
    top_3_zones = zone_revenue['salesofficename'].nlargest(3)
    trend_df = df[df['salesofficename'].isin(top_3_zones)]

```

```

trend_df['month_year'] = trend_df['billingdate']

zone_trend = trend_df.groupby(['month_year',
sns.lineplot(data=zone_trend, x='month_year',

ax3.set_title("Monthly Revenue Trends (Top 3 Z
ax3.set_xlabel("Timeline")
ax3.set_ylabel("Monthly Revenue")
ax3.legend(title="Zone")
else:
    ax3.text(0.5, 0.5, "Billing Date missing for 1

# -----
# CHART 4: Product Division Breakdown
# -----
ax4 = plt.subplot(2, 2, 4)
if 'divisiondesc' in df.columns:
    division_rev = df.groupby('divisiondesc')['net

    # Using a pie chart for market share visualiza
    ax4.pie(division_rev['netvalue'], labels=divi
    ax4.set_title("Revenue Market Share by Product
elif 'plantdesc' in df.columns:
    plant_rev = df.groupby('plantdesc')['netvalue
    sns.barplot(data=plant_rev, x='netvalue', y='t
    ax4.set_title("Top 5 Manufacturing Plants by F
    ax4.set_xlabel("Total Revenue")
    ax4.set_ylabel("")
else:
    ax4.text(0.5, 0.5, "Division/Plant Data Missin

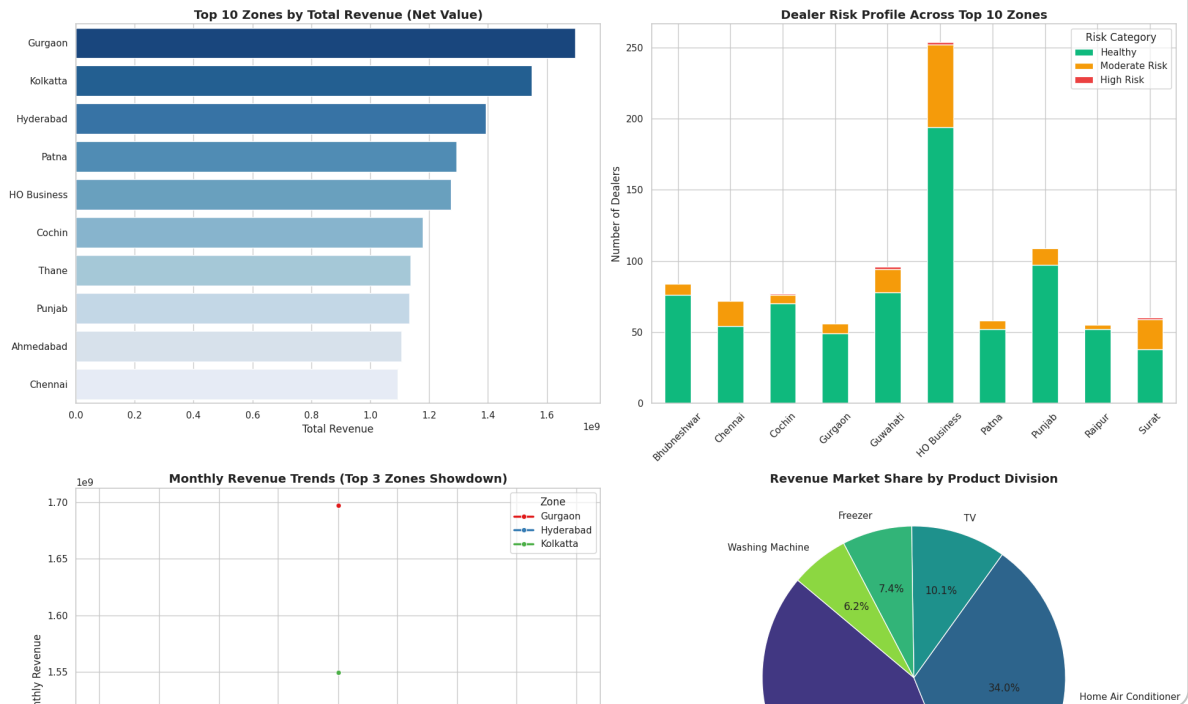
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
print("✅ ZONE-WISE VISUALIZATIONS COMPLETE.")

else:
    print("⚠️ Column 'salesofficename' not found. Canno

```

INITIATING ADVANCED ZONE-WISE VISUALIZATIONS...

Enterprise Zone-Wise (Sales Office) Performance Dashboard



```
print("🔍 ANALYZING DEALERS IN KOLKATTA...")

# Filter dealer profiles for 'Kolkatta' sales office
kolkatta_dealers = dealer_profiles[dealer_profiles['sales_office'] == 'Kolkatta']

if not kolkatta_dealers.empty:
    # Sort by health_score to find best and worst
    kolkatta_dealers_sorted = kolkatta_dealers.sort_values('health_score', ascending=False)

    print("\n🏆 BEST PERFORMING DEALERS IN KOLKATTA:")
    best_dealers = kolkatta_dealers_sorted.head(5)
    print(best_dealers[['customername', 'health_score', 'risk_category']])

    print("\nWorst PERFORMING DEALERS IN KOLKATTA:")
    worst_dealers = kolkatta_dealers_sorted.tail(5)
    print(worst_dealers[['customername', 'health_score', 'risk_category']])
else:
    print("No dealers found for 'Kolkatta' in the analysis.")
```

🔍 ANALYZING DEALERS IN KOLKATTA...

🏆 BEST PERFORMING DEALERS IN KOLKATTA:

customername	health_score	risk_category
K.M ENTERPRISE (HOWRAH)	82.09	Healthy
HINDUSTHAN DISTRIBUTOR (KRISHNAGAR)	79.56	Healthy
SKYLAB ELECTRONICS	77.96	Healthy
WORLD ELECTRONICS.	77.94	Healthy
MONDAL DISTRIBUTION PVT. LTD	77.86	Healthy

Worst PERFORMING DEALERS IN KOLKATTA:

customername	health_score	risk_category
--------------	--------------	---------------

:-----:	:-----:	:-----:
PINPOINT LOGISTICS	72.53	Moderate Risk
REFRIGERATION TRADE IMPEX	71.31	Moderate Risk
ANMOL ELECTRONICS.	70.49	Moderate Risk
ARATI ELECTRONICS (BONGAON)	62.92	Moderate Risk
KARUNA MANAGEMENT SERVICES LTD	38.88	High Risk

```
print("🔍 ANALYZING DEALERS IN EAST ZONE...")

# Define sales offices that belong to the 'East Zone'
EAST_ZONE_SALES_OFFICES = ['Kolkatta', 'Patna', 'Bhubr

# Map salesofficename to a broader 'zone' in dealer_p
# First, ensure salesofficename is available in dealer
if 'salesofficename' not in dealer_profiles.columns:
    # If 'salesofficename' was merged in HZG9gW05s04M,
    # If the previous cell didn't run, we need to ensu
    # For this current state, it should exist from HZC
    print("Warning: 'salesofficename' column not found
    customer_zone = df.groupby('customername')['saleso
    dealer_profiles = dealer_profiles.merge(customer_

dealer_profiles['zone'] = dealer_profiles['salesoffic

# Filter dealer profiles for the 'East Zone'
east_zone_dealers = dealer_profiles[dealer_profiles[':

if not east_zone_dealers.empty:
    # Sort by health_score to find best and worst
    east_zone_dealers_sorted = east_zone_dealers.sort

    print("\n🏆 BEST PERFORMING DEALERS IN EAST ZONE:
    best_dealers_ez = east_zone_dealers_sorted.head(5)
    print(best_dealers_ez[['customername', 'health_sc

    print("\nWorst PERFORMING DEALERS IN EAST ZONE:")
    worst_dealers_ez = east_zone_dealers_sorted.tail(5)
    print(worst_dealers_ez[['customername', 'health_sc
else:
    print("No dealers found for the defined 'East Zone"
```

🔍 ANALYZING DEALERS IN EAST ZONE...

🏆 BEST PERFORMING DEALERS IN EAST ZONE:

customername	health_score	risk_category	sa
:-----:	:-----:	:-----:	:--
ADITYA VISION LIMITED (PATNA)	87.98	Healthy	Pa
K.M ENTERPRISE (HOWRAH)	82.09	Healthy	Ko
HINDUSTHAN DISTRIBUTOR (KRISHNAGAR)	79.56	Healthy	Ko
SAAN TRADERS	78.71	Healthy	Bh
ATIK ENTERPRISES	78.67	Healthy	Pa

Worst PERFORMING DEALERS IN EAST ZONE:

customername	health_score	risk_category	salesof
:-----:	:-----:	:-----:	:-----

ARATI ELECTRONICS (BONGAON)	62.92	Moderate Risk	Kolkatt
MOBILE INDIA HOME APPLIANCES	60.2	Moderate Risk	Bhubnes
B S ENTERPRISES	58.33	Moderate Risk	Patna
SHREE DURLAVA	45.37	Moderate Risk	Bhubnes
KARUNA MANAGEMENT SERVICES LTD	38.88	High Risk	Kolkatt

```
print("🔍 ANALYZING DEALERS IN BHUBNESHWAR...")
```

```
# Filter dealer profiles for 'Bhubneshwar' sales office
```

```
bhubneshwar_dealers = dealer_profiles[dealer_profiles['salesofficename'] ==
```

```
if not bhubneshwar_dealers.empty:
```

```
    # Sort by health_score to find best and worst
```

```
    bhubneshwar_dealers_sorted = bhubneshwar_dealers.sort_values(by='health_
```

```
    print("\n🏆 BEST PERFORMING DEALERS IN BHUBNESHWAR:")
```

```
    best_dealers_bhubneshwar = bhubneshwar_dealers_sorted.head(5)
```

```
    print(best_dealers_bhubneshwar[['customername', 'health_score', 'risk_ca
```

```
    print("\nWorst PERFORMING DEALERS IN BHUBNESHWAR:")
```

```
    worst_dealers_bhubneshwar = bhubneshwar_dealers_sorted.tail(5)
```

```
    print(worst_dealers_bhubneshwar[['customername', 'health_score', 'risk_c
```

```
else:
```

```
    print("No dealers found for 'Bhubneshwar' in the analyzed profiles.")
```

```
🔍 ANALYZING DEALERS IN BHUBNESHWAR...
```

```
🏆 BEST PERFORMING DEALERS IN BHUBNESHWAR:
```

customername	health_score	risk_category	salesoffi
SAAN TRADERS	78.71	Healthy	Bhubneshw
ARPAN ENTERPRISES	78.38	Healthy	Bhubneshw
MAA SAKTI TRADERS (JAJPUR)	78.15	Healthy	Bhubneshw
GUPTA DISTRIBUTORS (CUTTACK)	78.13	Healthy	Bhubneshw
Patra Electronics	78.07	Healthy	Bhubneshw

```
Worst PERFORMING DEALERS IN BHUBNESHWAR:
```

customername	health_score	risk_category	sa
DREAM HOME	72.75	Moderate Risk	Bh
R.J. DISTRIBUTOR	72.72	Moderate Risk	Bh
SHREE GAJANANA ELECTRONICS AND FURN	63.02	Moderate Risk	Bh
MOBILE INDIA HOME APPLIANCES	60.2	Moderate Risk	Bh
SHREE DURLAVA	45.37	Moderate Risk	Bh

```
bold', y=0.98)

=False).head(10).reset_index()
    ax=axes[0, 0])
eight='bold')

nter', fontsize=12)

lums:

sort_values(ascending=False).head(10).reset_index()
eens_r', ax=axes[0, 1])
tweight='bold')

mpty else None
> 1 else None

a='center', fontsize=12)

_1_name]
alse).head(5)
urples_r', ax=axes[1, 0])
'bold')

va='center', fontsize=12)
```

```

_2_name]
else).head(5)
ranges_r', ax=axes[1, 1])
'bold')

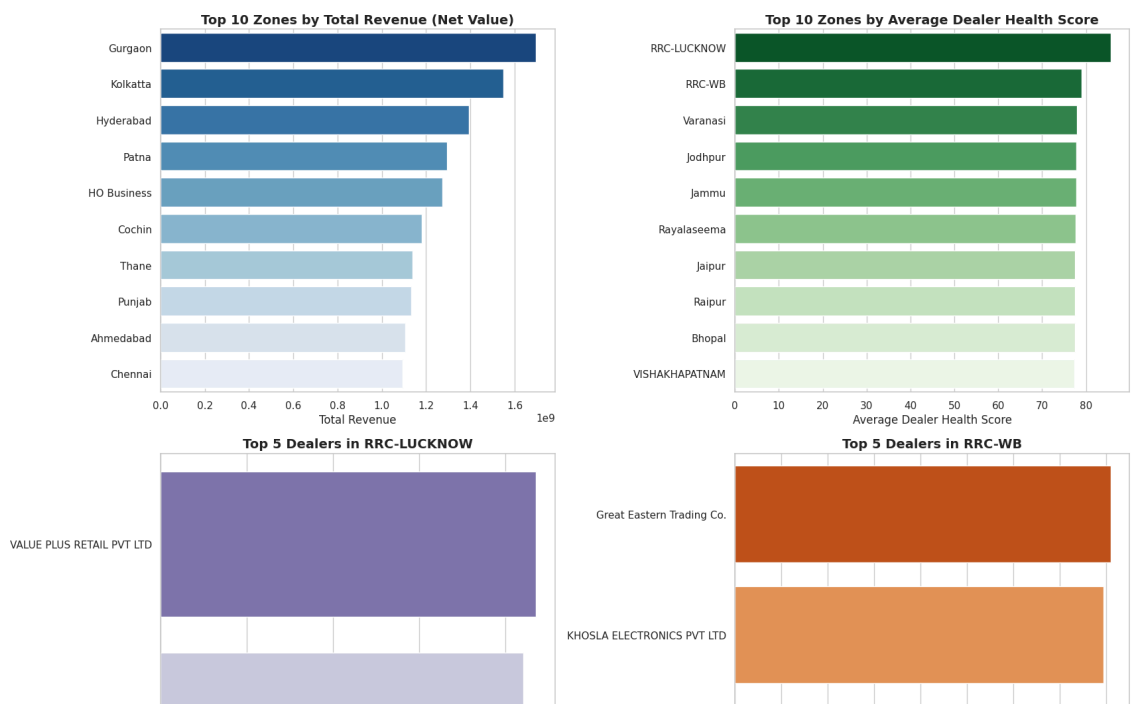
va='center', fontsize=12)

```



GENERATING ZONE AND DEALER PERFORMANCE VISUALIZATIONS...

Enterprise Zone & Dealer Performance Dashboard



```

# This project is an Enterprise Dealer Intelligence Dashboard
# It encompasses several key stages:
# 1. **Data Ingestion & Cleaning**: Loading raw sales data into a structured format.
# 2. **Exploratory Data Analysis (EDA)**: Understanding the distribution and trends of the data.
# 3. **Feature Engineering (Dealer Health Score)**: Calculating a composite score based on various dealer metrics.
# 4. **Machine Learning (Risk Classification)**: Training models to predict dealer risk levels.
# 5. **Time-Series Forecasting**: Utilizing a SARIMA model to forecast future sales and trends.
# 6. **Visualization Dashboard**: Generating interactive charts and maps for data exploration.

```

```

# This entire codebase is designed to provide a holistic view of dealer performance and risk.

```

```

# Below is the consolidated code for which you can create a Jupyter Notebook.

```

```

# ```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import io

```

```

# Machine Learning Libraries
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import LabelEncoder
import xgboost as xgb
from statsmodels.tsa.statespace.sarimax import SARIMAX

# Setup visualization styles
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (12, 6)

# print("🚀 INITIATING ENTERPRISE ML PIPELINE...\n")

# =====
# 0. FILE UPLOAD (GOOGLE COLAB NATIVE)
# =====
try:
    from google.colab import files
    # print("📁 Please click 'Choose Files' below and upload")
    # uploaded = files.upload()

    # # Automatically grab the exact name of the file
    # file_path = list(uploaded.keys())[0]
    # print(f"✅ Successfully uploaded: {file_path}\n")

    # # Read the file directly from the uploaded memory
    # df = pd.read_csv(io.BytesIO(uploaded[file_path]))
    # Placeholder for Streamlit - in a web app, file upload would be handled differently
    file_path = "filtered_data.csv" # Assume file is already in the environment
    df = pd.read_csv(file_path)

except ImportError:
    # Fallback just in case you run this on a local Jupyter environment
    file_path = "filtered_data.csv"
    # print(f"⚠️ Not in Google Colab. Looking for local file at {file_path}")
    df = pd.read_csv(file_path)

# =====
# 1. DATA CLEANING
# =====
# print("🕒 STEP 1: Cleaning Data...")

df.columns = df.columns.str.strip().str.lower()

# Clean Numeric Data
for col in ['invoicedqty', 'netvalue']:
    if col in df.columns:
        if df[col].dtype == 'object':
            df[col] = df[col].astype(str).str.replace(',', '')
            df[col] = pd.to_numeric(df[col], errors='coerce')

# Clean Dates

```

```

if 'billingdate' in df.columns:
    df['billingdate'] = pd.to_datetime(df['billingdate'])

# print(f"✅ Data Cleaned: {df.shape[0]:,} rows ready")

# =====
# 2. FEATURE ENGINEERING (DEALER HEALTH SCORE)
# =====
# print("⌚ STEP 2: Engineering Dealer Health Profile")
dealer_profiles = df.groupby('customername').agg(
    total_transactions=('billingdocument', 'count'),
    total_volume=('invoicedqty', 'sum'),
    total_revenue=('netvalue', 'sum'),
    total_returns=('invoicedqty', lambda x: (x < 0).sum())
).reset_index()

# Calculate Metrics
dealer_profiles['return_rate_pct'] = (dealer_profiles['total_returns'] / dealer_profiles['total_transactions']) * 100
dealer_profiles['avg_transaction_val'] = dealer_profiles['total_revenue'] / dealer_profiles['total_transactions']
dealer_profiles = dealer_profiles[dealer_profiles['total_transactions'] > 0]

# Calculate Raw Score
rev_max, vol_max = dealer_profiles['total_revenue'].max(), dealer_profiles['total_volume'].max()
dealer_profiles['raw_score'] = (
    ((dealer_profiles['total_revenue'] / rev_max) * 40) +
    ((dealer_profiles['total_volume'] / vol_max) * 40) +
    (dealer_profiles['return_rate_pct'] * 2.5)
)

# Normalize 0-100
min_score, max_score = dealer_profiles['raw_score'].min(), dealer_profiles['raw_score'].max()
dealer_profiles['health_score'] = 100 * (dealer_profiles['raw_score'] - min_score) / (max_score - min_score)

# Classify Risk
def assign_risk(score):
    if score >= 75: return 'Healthy'
    elif score >= 45: return 'Moderate Risk'
    else: return 'High Risk'

dealer_profiles['risk_category'] = dealer_profiles['health_score'].apply(assign_risk)
# print("✅ Health Scores and Risk Categories Generated")

# =====
# 3. MACHINE LEARNING (RISK CLASSIFICATION)
# =====
# print("⌚ STEP 3: Training ML Classification Models")

features = ['total_transactions', 'total_volume', 'total_revenue', 'return_rate_pct']
X = dealer_profiles[features]
y = dealer_profiles['risk_category']

# Encode the text labels for XGBoost
le = LabelEncoder()
y_encoded = le.fit_transform(y)

```

```

X_train, X_test, y_train, y_test = train_test_split(X,
# Random Forest
rf_model = RandomForestClassifier(n_estimators=100, r
rf_model.fit(X_train, y_train)
rf_preds = rf_model.predict(X_test)

# XGBoost
xgb_model = xgb.XGBClassifier(eval_metric='mlogloss',
xgb_model.fit(X_train, y_train)
xgb_preds = xgb_model.predict(X_test)

# print("🏆 ML MODEL RESULTS:")
# print(f"    • Random Forest Accuracy : {accuracy_scor
# print(f"    • XGBoost Accuracy       : {accuracy_scor

# =====
# 4. TIME-SERIES FORECASTING (SARIMAX)
# =====
# print("🕒 STEP 4: Running Network Volume Forecast..

clean_dates_df = df.dropna(subset=['billingdate']).co
clean_dates_df['month_year'] = clean_dates_df['billing
network_data = clean_dates_df.groupby('month_year').a

if len(network_data) >= 4:
    train, test = network_data.iloc[:-2], network_data
    arima_model = SARIMAX(train['invoicedqty'], order=
    arima_fit = arima_model.fit(dispatch=False)
    arima_preds = np.maximum(0, np.round(arima_fit.pro
    # print("✅ SARIMAX Forecast Complete.\n")
else:
    # print("⚠️ Not enough months for forecasting.\n"
    pass # No forecast if not enough data

# =====
# 5. VISUALIZATION DASHBOARD (JUPYTER OUTPUT)
# =====
# print("📊 STEP 5: Generating Visualizations...\n")

fig = plt.figure(figsize=(18, 12))
fig.suptitle("Enterprise Dealer Intelligence Dashboard")

# Chart 1: Risk Distribution
ax1 = plt.subplot(2, 2, 1)
sns.countplot(data=dealer_profiles, x='risk_category',
ax1.set_title("Dealer Risk Distribution")
ax1.set_ylabel("Number of Dealers")

# Chart 2: Feature Importance (XGBoost)
ax2 = plt.subplot(2, 2, 2)
importances = xgb_model.feature_importances_
imp_df = pd.DataFrame({'Feature': features, 'Importanc
sns.barplot(data=imp_df, x='Importance', y='Feature',
ax2.set_title("What Drives Dealer Risk? (Feature Impor

```

```

# Chart 3: Health Score vs Return Rate
ax3 = plt.subplot(2, 2, 3)
sns.scatterplot(data=dealer_profiles, x='return_rate', y='health_score')
ax3.set_title("Impact of Returns on Health Score")
ax3.set_xlabel("Return Rate (%)")
ax3.set_ylabel("Health Score (0-100)")

# Chart 4: Forecast
ax4 = plt.subplot(2, 2, 4)
if len(network_data) >= 4:
    ax4.plot(network_data['month_year'], network_data['volume'], label='Actual')
    ax4.plot(test['month_year'], arima_preds, label='Forecast')
    ax4.set_title("Enterprise Volume Forecast (Net of 1000000)")
    ax4.legend()
else:
    ax4.text(0.5, 0.5, "Not enough months for forecast")

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

# print("✅ PIPELINE EXECUTION COMPLETE.")

# Advanced Zone-Wise Visualizations
# print("🚀 INITIATING ADVANCED ZONE-WISE VISUALIZATIONS")

# =====
# 1. MAP DEALERS TO THEIR PRIMARY ZONE
# =====
if 'salesofficename' in df.columns:
    customer_zone = df.groupby('customername')['salesofficename'].first()

    if 'salesofficename' not in dealer_profiles.columns:
        dealer_profiles = dealer_profiles.merge(customer_zone, left_index=True, right_index=True)

    # =====
    # 2. GENERATE THE ZONE-WISE DASHBOARD
    # =====
    fig = plt.figure(figsize=(20, 16))
    fig.suptitle("🌐 Enterprise Zone-Wise (Sales Office) Analysis")

    # -----
    # CHART 1: Top 10 Zones by Total Revenue
    # -----
    ax1 = plt.subplot(2, 2, 1)
    zone_revenue = df.groupby('salesofficename')['netvalue'].sum()
    sns.barplot(data=zone_revenue, x='salesofficename', y='netvalue')
    ax1.set_title("Top 10 Zones by Total Revenue (Net of 1000000)")
    ax1.set_xlabel("Total Revenue")
    ax1.set_ylabel("")

    # -----
    # CHART 2: Dealer Risk Distribution by Top Zones
    # -----
    ax2 = plt.subplot(2, 2, 2)

```

```

top_zones_by_dealers = dealer_profiles['salesofficename'].value_counts()
filtered_profiles = dealer_profiles[dealer_profiles['salesofficename'].isin(top_zones_by_dealers)]

risk_crosstab = pd.crosstab(filtered_profiles['salesofficename'], filtered_profiles['riskcategory'])
for col in ['Healthy', 'Moderate Risk', 'High Risk']:
    if col not in risk_crosstab.columns: risk_crosstab[col] = 0

risk_crosstab = risk_crosstab[['Healthy', 'Moderate Risk', 'High Risk']]
risk_crosstab.plot(kind='bar', stacked=True, color=['#2ca02c', '#d62728', '#17becf'])

ax2.set_title("Dealer Risk Profile Across Top 10 Zones")
ax2.set_xlabel("")
ax2.set_ylabel("Number of Dealers")
ax2.tick_params(axis='x', rotation=45)
ax2.legend(title="Risk Category")

# -----
# CHART 3: Monthly Revenue Trend for Top 3 Zones
# -----
ax3 = plt.subplot(2, 2, 3)
if 'billingdate' in df.columns:
    top_3_zones = zone_revenue['salesofficename'].value_counts().head(3)
    trend_df = df[df['salesofficename'].isin(top_3_zones)]
    trend_df['month_year'] = trend_df['billingdate'].dt.to_period('M').strftime('%Y-%m')

    zone_trend = trend_df.groupby(['month_year', 'salesofficename']).sum('netvalue')
    sns.lineplot(data=zone_trend, x='month_year', y='netvalue', style='salesofficename')

    ax3.set_title("Monthly Revenue Trends (Top 3 Zones)")
    ax3.set_xlabel("Timeline")
    ax3.set_ylabel("Monthly Revenue")
    ax3.legend(title="Zone")
else:
    ax3.text(0.5, 0.5, "Billing Date missing for Top 3 Zones")

# -----
# CHART 4: Product Division Breakdown
# -----
ax4 = plt.subplot(2, 2, 4)
if 'divisiondesc' in df.columns:
    division_rev = df.groupby('divisiondesc')['netvalue'].sum().sort_values(ascending=False).head(5)

    ax4.pie(division_rev['netvalue'], labels=division_rev['divisiondesc'], autopct='%1.1f%%')
    ax4.set_title("Revenue Market Share by Product Division")
elif 'plantdesc' in df.columns:
    plant_rev = df.groupby('plantdesc')['netvalue'].sum().sort_values(ascending=False).head(5)
    sns.barplot(data=plant_rev, x='netvalue', y='plantdesc')
    ax4.set_title("Top 5 Manufacturing Plants by Revenue")
    ax4.set_xlabel("Total Revenue")
    ax4.set_ylabel("")
else:
    ax4.text(0.5, 0.5, "Division/Plant Data Missing")

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

```

    # print("✅ ZONE-WISE VISUALIZATIONS COMPLETE.")

else:
    # print("⚠️ Column 'salesofficename' not found. C
    pass # No zone-wise charts if salesofficename is n

# Zone and Dealer Performance Visualizations
# print("📊 GENERATING ZONE AND DEALER PERFORMANCE VI

fig, axes = plt.subplots(2, 2, figsize=(18, 14))
fig.suptitle("Enterprise Zone & Dealer Performance Da

# =====
# CHART 1: Top 10 Zones by Total Revenue
# (To identify best performing zones based on direct i
# =====
if 'salesofficename' in df.columns and 'netvalue' in c
    zone_revenue = df.groupby('salesofficename')['netv
    sns.barplot(data=zone_revenue, x='netvalue', y='sa
    axes[0, 0].set_title("Top 10 Zones by Total Revenu
    axes[0, 0].set_xlabel("Total Revenue")
    axes[0, 0].set_ylabel("")
else:
    axes[0, 0].text(0.5, 0.5, "Data Missing for Zone f

# =====
# CHART 2: Top 10 Zones by Average Dealer Health Score
# (To identify best performing zones based on overall
# =====
if 'salesofficename' in dealer_profiles.columns and 'h
    zone_avg_health = dealer_profiles.groupby('salesof
    sns.barplot(data=zone_avg_health, x='health_score'
    axes[0, 1].set_title("Top 10 Zones by Average Dea
    axes[0, 1].set_xlabel("Average Dealer Health Score
    axes[0, 1].set_ylabel("")

    top_zone_1_name = zone_avg_health.loc[0, 'salesoft
    top_zone_2_name = zone_avg_health.loc[1, 'salesoft
else:
    axes[0, 1].text(0.5, 0.5, "Data Missing for Zone f
    top_zone_1_name = None
    top_zone_2_name = None

# =====
# CHART 3: Top 5 Dealers in the Overall Best Performin
# (Drill-down for the best zone by average dealer hea
# =====
if top_zone_1_name:
    best_zone_1_dealers = dealer_profiles[dealer_profi
    best_zone_1_dealers = best_zone_1_dealers.sort_va
    sns.barplot(data=best_zone_1_dealers, x='health_sc
    axes[1, 0].set_title(f"Top 5 Dealers in {top_zone_
    axes[1, 0].set_xlabel("Dealer Health Score")
    axes[1, 0].set_ylabel("")
else:

```

```

    axes[1, 0].text(0.5, 0.5, "Cannot display Top Dea

# =====
# CHART 4: Top 5 Dealers in the Second Best Performing
# (Drill-down for the second best zone by average dea
# =====
if top_zone_2_name:
    best_zone_2_dealers = dealer_profiles[dealer_prof:
    best_zone_2_dealers = best_zone_2_dealers.sort_va
    sns.barplot(data=best_zone_2_dealers, x='health_sc
    axes[1, 1].set_title(f"Top 5 Dealers in {top_zone_
    axes[1, 1].set_xlabel("Dealer Health Score")
    axes[1, 1].set_ylabel("")
else:
    axes[1, 1].text(0.5, 0.5, "Cannot display Top Dea

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
# print("✅ ZONE AND DEALER VISUALIZATIONS COMPLETE."

# Analysis of dealers in specific zones
# print("🔍 ANALYZING DEALERS IN KOLKATTA...")
# kolkatta_dealers = dealer_profiles[dealer_profiles[
# if not kolkatta_dealers.empty:
#     kolkatta_dealers_sorted = kolkatta_dealers.sort_
# print("\n🏆 BEST PERFORMING DEALERS IN KOLKATTA:")
# best_dealers = kolkatta_dealers_sorted.head(5)
# print(best_dealers[['customername', 'health_score',
# print("\nWorst PERFORMING DEALERS IN KOLKATTA:")
# worst_dealers = kolkatta_dealers_sorted.tail(5)
# print(worst_dealers[['customername', 'health_score',
# else:
# print("No dealers found for 'Kolkatta' in the analy
# pass # No dealers in Kolkatta

# East Zone Analysis
# print("🔍 ANALYZING DEALERS IN EAST ZONE...")
# EAST_ZONE_SALES_OFFICES = ['Kolkatta', 'Patna', 'Bhu
# if 'salesofficename' not in dealer_profiles.columns:
# print("Warning: 'salesofficename' column not found :
# customer_zone = df.groupby('customername')['salesoff
# dealer_profiles = dealer_profiles.merge(customer_zor
# dealer_profiles['zone'] = dealer_profiles['salesoff:
# east_zone_dealers = dealer_profiles[dealer_profiles[
# if not east_zone_dealers.empty:
# east_zone_dealers_sorted = east_zone_dealers.sort_v
# print("\n🏆 BEST PERFORMING DEALERS IN EAST ZONE:")
# best_dealers_ez = east_zone_dealers_sorted.head(5)
# print(best_dealers_ez[['customername', 'health_score
# print("\nWorst PERFORMING DEALERS IN EAST ZONE:")
# worst_dealers_ez = east_zone_dealers_sorted.tail(5)
# print(worst_dealers_ez[['customername', 'health_scoi
# else:

```

```

# print("No dealers found for the defined 'East Zone'")
# pass # No dealers in East Zone

# Bhubneshwar Zone Analysis
# print("🔍 ANALYZING DEALERS IN BHUBNESHWAR...")
# bhubneshwar_dealers = dealer_profiles[dealer_profile
# if not bhubneshwar_dealers.empty:
# bhubneshwar_dealers_sorted = bhubneshwar_dealers.sort
# print("\n🏆 BEST PERFORMING DEALERS IN BHUBNESHWAR:
# best_dealers_bhubneshwar = bhubneshwar_dealers_sort
# print(best_dealers_bhubneshwar[['customername', 'hea
# print("\nWorst PERFORMING DEALERS IN BHUBNESHWAR:")
# worst_dealers_bhubneshwar = bhubneshwar_dealers_sort
# print(worst_dealers_bhubneshwar[['customername', 'hea
# else:
# print("No dealers found for 'Bhubneshwar' in the an
# pass # No dealers in Bhubneshwar

# ```

# ```python
# Consolidated Zone-wise Top and Worst Dealers Analys
# print("🔍 ANALYZING TOP AND WORST DEALERS FOR ALL Z

if 'salesofficename' in df.columns:
    # Ensure dealer_profiles has 'salesofficename' mer
    if 'salesofficename' not in dealer_profiles.columns:
        customer_zone = df.groupby('customername')['sa
        dealer_profiles = dealer_profiles.merge(custor

    unique_sales_offices = dealer_profiles['salesoffic

    for sales_office in unique_sales_offices:
        # print(f"\n--- Analyzing Dealers in Zone: {sa
        zone_dealers = dealer_profiles[dealer_profiles

        if not zone_dealers.empty:
            zone_dealers_sorted = zone_dealers.sort_va

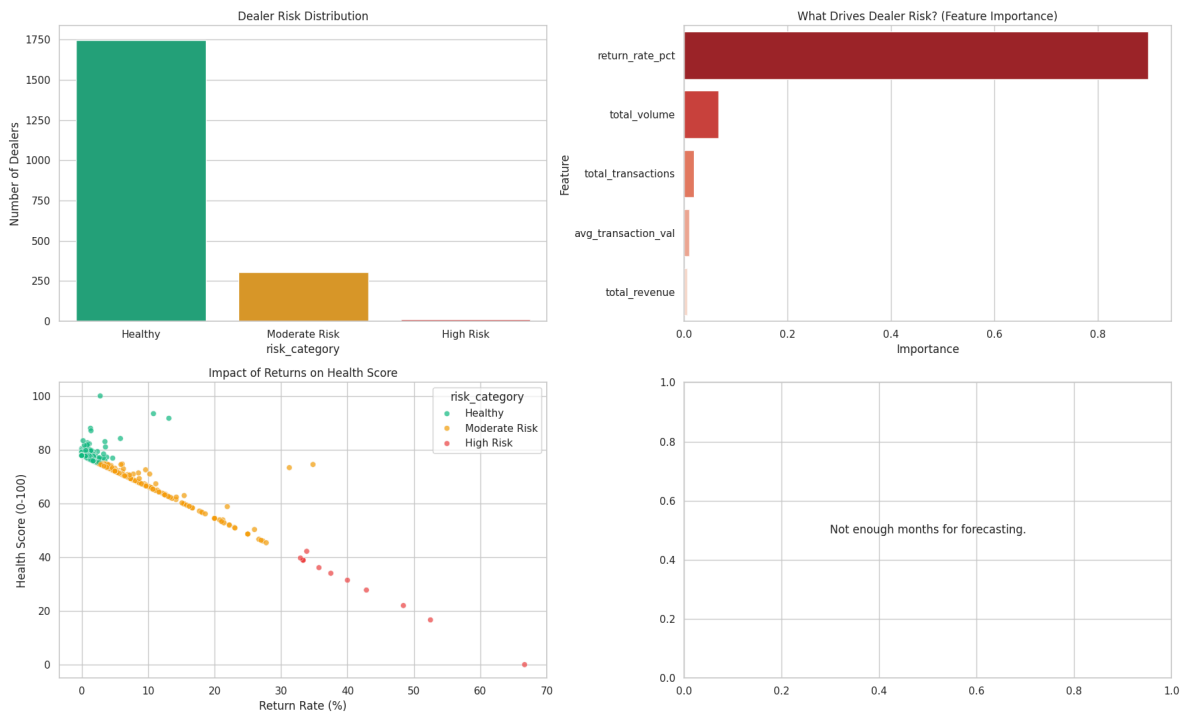
            # print(f"\n🏆 BEST PERFORMING DEALERS IN
            best_dealers = zone_dealers_sorted.head(5)
            # print(best_dealers[['customername', 'hea

            # print(f"\nWorst PERFORMING DEALERS IN {s
            worst_dealers = zone_dealers_sorted.tail(5)
            # print(worst_dealers[['customername', 'hea
        # else:
            # print(f"No dealers found for '{sales_offi

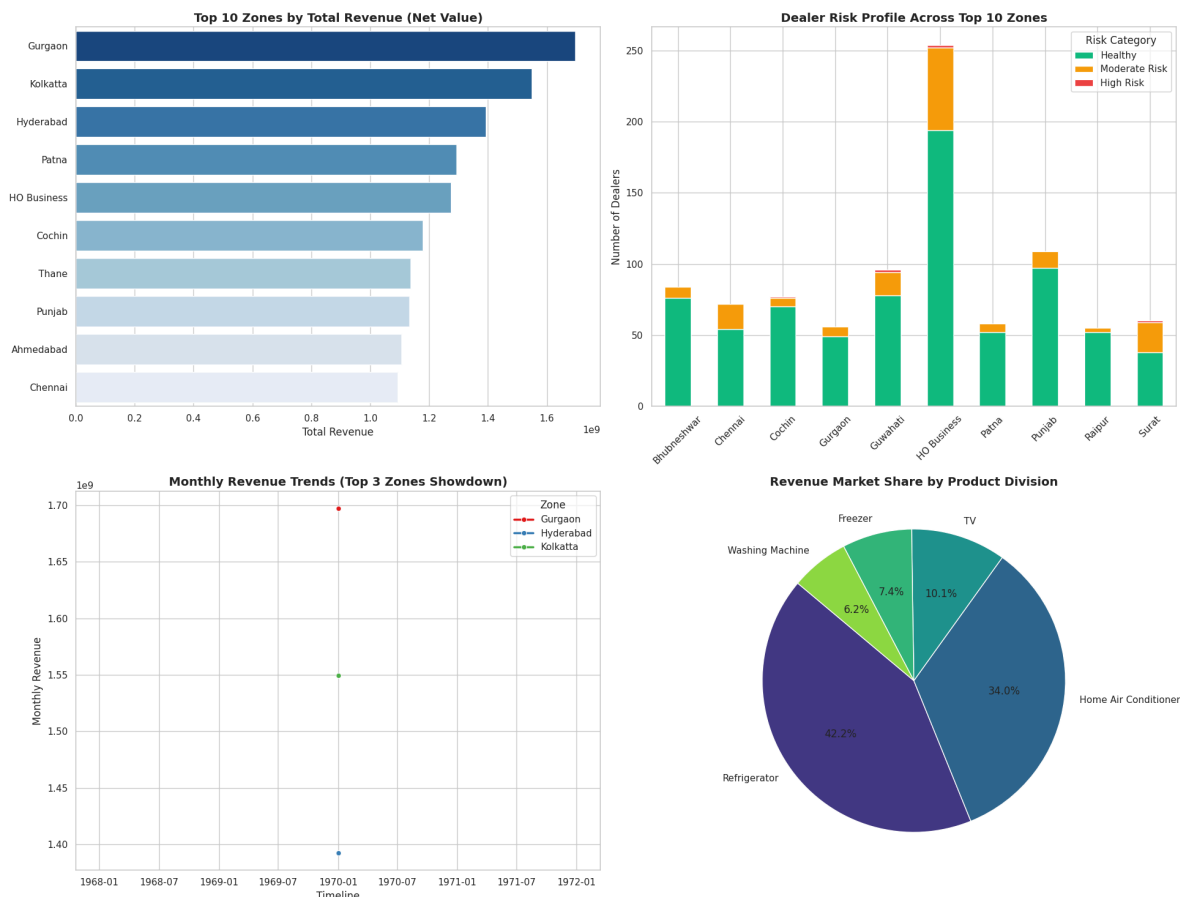
# else:
    # print("⚠️ Column 'salesofficename' not found. Can
# ```

```

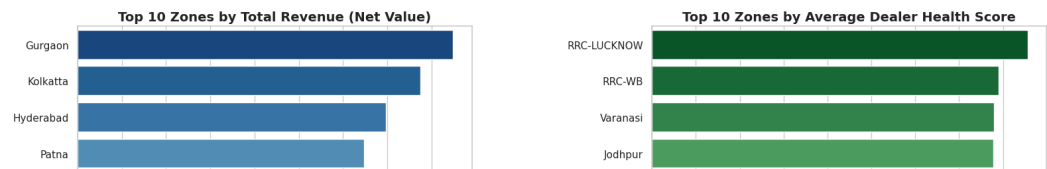
Enterprise Dealer Intelligence Dashboard



Enterprise Zone-Wise (Sales Office) Performance Dashboard



Enterprise Zone & Dealer Performance Dashboard



```

import streamlit as st
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
import seaborn as sns

# ML & Forecasting Libraries
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import LabelEncoder
import xgboost as xgb
from statsmodels.tsa.statespace.sarimax import SARIMAX

# Setup visualization styles
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (12, 6)

def main():
    st.set_page_config(layout="wide", page_title="Enterprise Dealer Intelligence Dashboard")

    st.title("🚀 Enterprise Dealer Intelligence Dashboard")
    st.markdown("This dashboard provides insights into dealer performance and market trends.")

    # =====
    # 0. FILE UPLOAD
    # =====
    st.header("1. Upload Your Data")
    uploaded_file = st.file_uploader("Choose a CSV file")

    if uploaded_file is not None:
        df = pd.read_csv(uploaded_file)
        st.success("✅ File uploaded successfully!")
        st.dataframe(df.head())

    # =====
    # 1. DATA CLEANING
    # =====
    st.header("2. Data Cleaning")
    df.columns = df.columns.str.strip().str.lower()

    # Clean Numeric Data
    for col in ['invoicedqty', 'netvalue']:
        if col in df.columns:
            if df[col].dtype == 'object':
                df[col] = df[col].astype(str).str.replace(',', '')
                df[col] = pd.to_numeric(df[col], errors='coerce')

    # Clean Dates
    if 'billingdate' in df.columns:
        df['billingdate'] = pd.to_datetime(df['billingdate'], errors='coerce')

    st.write(f"✅ Data Cleaned: {df.shape[0]:,} rows")

```

```

st.dataframe(df.head())

# =====
# 2. FEATURE ENGINEERING (DEALER HEALTH SCORE)
# =====
st.header("3. Dealer Health Score Engineering")
dealer_profiles = df.groupby('customername').agg(
    total_transactions=('billingdocument', 'count'),
    total_volume=('invoicedqty', 'sum'),
    total_revenue=('netvalue', 'sum'),
    total_returns=('invoicedqty', lambda x: x.sum())
).reset_index()

# Calculate Metrics
dealer_profiles['return_rate_pct'] = (dealer_profiles['total_returns'] / dealer_profiles['total_transactions']) * 100
dealer_profiles['avg_transaction_val'] = dealer_profiles['total_revenue'] / dealer_profiles['total_transactions']
dealer_profiles = dealer_profiles[dealer_profiles['total_transactions'] > 0]

# Calculate Raw Score
rev_max, vol_max = dealer_profiles['total_revenue'].max(), dealer_profiles['total_volume'].max()
dealer_profiles['raw_score'] = (
    ((dealer_profiles['total_revenue'] / rev_max) * 0.5) +
    ((dealer_profiles['total_volume'] / vol_max) * 0.5) +
    (dealer_profiles['return_rate_pct'] * 2.5)
)

# Normalize 0-100
min_score, max_score = dealer_profiles['raw_score'].min(), dealer_profiles['raw_score'].max()
dealer_profiles['health_score'] = 100 * (dealer_profiles['raw_score'] - min_score) / (max_score - min_score)

# Classify Risk
def assign_risk(score):
    if score >= 75: return 'Healthy'
    elif score >= 45: return 'Moderate Risk'
    else: return 'High Risk'

dealer_profiles['risk_category'] = dealer_profiles['health_score'].apply(assign_risk)
st.write("✅ Health Scores and Risk Categories")
st.dataframe(dealer_profiles[['customername', 'total_transactions', 'total_volume', 'total_revenue', 'total_returns', 'return_rate_pct', 'raw_score', 'health_score', 'risk_category']])

# =====
# 3. MACHINE LEARNING (RISK CLASSIFICATION)
# =====
st.header("4. ML Risk Classification")
features = ['total_transactions', 'total_volume', 'total_revenue', 'total_returns']
X = dealer_profiles[features]
y = dealer_profiles['risk_category']

le = LabelEncoder()
y_encoded = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2, random_state=42)

rf_model = RandomForestClassifier(n_estimators=100)
rf_model.fit(X_train, y_train)

```

```

rf_preds = rf_model.predict(X_test)

xgb_model = xgb.XGBClassifier(eval_metric='mlogloss')
xgb_model.fit(X_train, y_train)
xgb_preds = xgb_model.predict(X_test)

st.write("🏆 ML MODEL RESULTS:")
st.write(f"    • Random Forest Accuracy : {accuracy_score(y_test, rf_preds)}")
st.write(f"    • XGBoost Accuracy          : {accuracy_score(y_test, xgb_preds)}")

# =====
# 4. TIME-SERIES FORECASTING (SARIMAX)
# =====
st.header("5. Network Volume Forecasting")
clean_dates_df = df.dropna(subset=['billingdate'])
clean_dates_df['month_year'] = clean_dates_df['billingdate'].dt.strftime('%Y-%m')
network_data = clean_dates_df.groupby('month_year').sum().reset_index()

if len(network_data) >= 4:
    train, test = network_data.iloc[:-2], network_data.iloc[-2:]
    arima_model = SARIMAX(train['invoicedqty'], order=(1,1,1))
    arima_fit = arima_model.fit(dispatch=False)
    arima_preds = np.maximum(0, np.round(arima_fit.predict(len(train), len(train)+len(test))))
    st.write("✅ SARIMAX Forecast Complete.")
else:
    st.warning("⚠️ Not enough months for forecasting")

# =====
# 5. VISUALIZATION DASHBOARD
# =====
st.header("6. Visualization Dashboard")

st.subheader("Enterprise Dealer Intelligence Dashboard")
fig1, axes1 = plt.subplots(2, 2, figsize=(18, 12))

# Chart 1: Risk Distribution
sns.countplot(data=dealer_profiles, x='risk_category')
axes1[0, 0].set_title("Dealer Risk Distribution")
axes1[0, 0].set_ylabel("Number of Dealers")

# Chart 2: Feature Importance (XGBoost)
importances = xgb_model.feature_importances_
imp_df = pd.DataFrame({'Feature': features, 'Importance': importances})
sns.barplot(data=imp_df, x='Importance', y='Feature')
axes1[0, 1].set_title("What Drives Dealer Risk")

# Chart 3: Health Score vs Return Rate
sns.scatterplot(data=dealer_profiles, x='return_rate', y='health_score')
axes1[1, 0].set_title("Impact of Returns on Health")
axes1[1, 0].set_xlabel("Return Rate (%)")
axes1[1, 0].set_ylabel("Health Score (0-100)")

# Chart 4: Forecast
if len(network_data) >= 4:
    axes1[1, 1].plot(network_data['month_year'], network_data['invoicedqty'])

```

```

axes1[1, 1].plot(test['month_year'], arima
axes1[1, 1].set_title("Enterprise Volume I
axes1[1, 1].legend()
else:
    axes1[1, 1].text(0.5, 0.5, "Not enough mor

plt.tight_layout(rect=[0, 0, 1, 0.95])
st.pyplot(fig1)
plt.close(fig1) # Close the figure to prevent

# Advanced Zone-Wise Visualizations
st.subheader("🌐 Enterprise Zone-Wise (Sales

if 'salesofficename' in df.columns:
    customer_zone = df.groupby('customername')
    if 'salesofficename' not in dealer_profiles
        dealer_profiles = dealer_profiles.mer

fig2, axes2 = plt.subplots(2, 2, figsize=

# CHART 1: Top 10 Zones by Total Revenue
zone_revenue = df.groupby('salesofficename
sns.barplot(data=zone_revenue, x='netvalue
axes2[0, 0].set_title("Top 10 Zones by Tot
axes2[0, 0].set_xlabel("Total Revenue")
axes2[0, 0].set_ylabel("")

# CHART 2: Dealer Risk Distribution by Top
top_zones_by_dealers = dealer_profiles['sa
filtered_profiles = dealer_profiles[dealer

risk_crosstab = pd.crosstab(filtered_profi
for col in ['Healthy', 'Moderate Risk', 'H
    if col not in risk_crosstab.columns:
risk_crosstab = risk_crosstab[['Healthy',
risk_crosstab.plot(kind='bar', stacked=Tru

axes2[0, 1].set_title("Dealer Risk Profile
axes2[0, 1].set_xlabel("")
axes2[0, 1].set_ylabel("Number of Dealers'
axes2[0, 1].tick_params(axis='x', rotation
axes2[0, 1].legend(title="Risk Category")

# CHART 3: Monthly Revenue Trend for Top 3
if 'billingdate' in df.columns:
    top_3_zones = zone_revenue['salesoffic
    trend_df = df[df['salesofficename'].is
    trend_df['month_year'] = trend_df['bi

    zone_trend = trend_df.groupby(['month_
    sns.lineplot(data=zone_trend, x='month

    axes2[1, 0].set_title("Monthly Revenue
    axes2[1, 0].set_xlabel("Timeline")

```

```

axes2[1, 0].set_ylabel("Monthly Revenue")
axes2[1, 0].legend(title="Zone")
else:
    axes2[1, 0].text(0.5, 0.5, "Billing Data")

# CHART 4: Product Division Breakdown
if 'divisiondesc' in df.columns:
    division_rev = df.groupby('divisiondesc')['netvalue'].sum()
    axes2[1, 1].pie(division_rev['netvalue'])
    axes2[1, 1].set_title("Revenue Market Share by Division")
elif 'plantdesc' in df.columns:
    plant_rev = df.groupby('plantdesc')['netvalue'].sum()
    sns.barplot(data=plant_rev, x='netvalue', y='plantdesc')
    axes2[1, 1].set_title("Top 5 Manufacturing Plants")
    axes2[1, 1].set_xlabel("Total Revenue")
    axes2[1, 1].set_ylabel("")
else:
    axes2[1, 1].text(0.5, 0.5, "Division/Plant Revenue")

plt.tight_layout(rect=[0, 0, 1, 0.96])
st.pyplot(fig2)
plt.close(fig2)

# Zone and Dealer Performance Visualization
st.subheader("Enterprise Zone & Dealer Performance")
fig3, axes3 = plt.subplots(2, 2, figsize=(12, 10))

# CHART 1: Top 10 Zones by Total Revenue
zone_revenue = df.groupby('salesofficename')['netvalue'].sum()
sns.barplot(data=zone_revenue, x='netvalue', y='salesofficename')
axes3[0, 0].set_title("Top 10 Zones by Total Revenue")
axes3[0, 0].set_xlabel("Total Revenue")
axes3[0, 0].set_ylabel("")

# CHART 2: Top 10 Zones by Average Dealer Health
zone_avg_health = dealer_profiles.groupby('salesofficename')['healthscore'].mean()
sns.barplot(data=zone_avg_health, x='healthscore', y='salesofficename')
axes3[0, 1].set_title("Top 10 Zones by Average Dealer Health")
axes3[0, 1].set_xlabel("Average Dealer Health Score")
axes3[0, 1].set_ylabel("")

top_zone_1_name = zone_avg_health.loc[0, 'salesofficename']
top_zone_2_name = zone_avg_health.loc[1, 'salesofficename']

# CHART 3: Top 5 Dealers in the Overall Best Health
if top_zone_1_name:
    best_zone_1_dealers = dealer_profiles[dealer_profiles['salesofficename'] == top_zone_1_name]
    best_zone_1_dealers = best_zone_1_dealers[['dealerid', 'healthscore']]
    sns.barplot(data=best_zone_1_dealers, x='healthscore', y='dealerid')
    axes3[1, 0].set_title(f"Top 5 Dealers in {top_zone_1_name}")
    axes3[1, 0].set_xlabel("Dealer Health Score")
    axes3[1, 0].set_ylabel("")
else:
    axes3[1, 0].text(0.5, 0.5, "Cannot display data for this zone")

```

```

# CHART 4: Top 5 Dealers in the Second Best Zone
if top_zone_2_name:
    best_zone_2_dealers = dealer_profiles[dealer_profiles['zone'] == top_zone_2_name]
    best_zone_2_dealers = best_zone_2_dealers[['dealername', 'netvalue']]
    sns.barplot(data=best_zone_2_dealers, x='dealername', y='netvalue')
    axes3[1, 1].set_title(f"Top 5 Dealers in {top_zone_2_name}")
    axes3[1, 1].set_xlabel("Dealer Health")
    axes3[1, 1].set_ylabel("Net Value")
else:
    axes3[1, 1].text(0.5, 0.5, "Cannot display chart for this zone")

plt.tight_layout(rect=[0, 0, 1, 0.96])
st.pyplot(fig3)
plt.close(fig3)

st.subheader("Consolidated Zone-wise Top 5 Dealers")
unique_sales_offices = dealer_profiles['salesofficename'].unique()

for sales_office in unique_sales_offices:
    st.markdown(f"#### --- Analyzing Dealers in {sales_office}")
    zone_dealers = dealer_profiles[dealer_profiles['salesofficename'] == sales_office]

    if not zone_dealers.empty:
        zone_dealers_sorted = zone_dealers.sort_values('netvalue', ascending=False)

        st.markdown(f"**🏆 BEST PERFORMING DEALERS**")
        best_dealers = zone_dealers_sorted.head(5)
        st.dataframe(best_dealers[['dealername', 'netvalue']])

        st.markdown(f"**Worst PERFORMING DEALERS**")
        worst_dealers = zone_dealers_sorted.tail(5)
        st.dataframe(worst_dealers[['dealername', 'netvalue']])
    else:
        st.write(f"No dealers found for '{sales_office}'")

else:
    st.warning("⚠️ Column 'salesofficename' not found in dataset")

else:
    st.info("Please upload a CSV file to proceed with the analysis")

if __name__ == "__main__":
    main()

# To run this Streamlit app:
# 1. Save the code above as a Python file, e.g., `streamlit_app.py`
# 2. Open your terminal or command prompt.
# 3. Navigate to the directory where you saved the file.
# 4. Run the command: `streamlit run streamlit_app.py`
# If Streamlit is not installed, install it first:

# Note: This code assumes 'filtered_data.csv' is the primary dataset.
# The columns 'billingdate', 'invoicedqty', 'netvalue', 'customername', 'billingdocument', 'divisiondesc', 'productcode' are used
# in your dataset for the respective functionalities to

```

```
Collecting streamlit
```

```
  Downloading streamlit-1.58.0-py3-none-any.whl.metadata (9.6 kB)
```

```
Requirement already satisfied: altair!=5.4.0,!5.4.1,<7,>=4.0 in /usr/local/lib/python3.12
```

```
Requirement already satisfied: blinker<2,>=1.5.0 in /usr/local/lib/python3.12
```

```
Requirement already satisfied: cachetools<8,>=5.5 in /usr/local/lib/python3.12
```

```
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.12/dis
```

```
Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in /usr/local/lib
```

```
Requirement already satisfied: numpy<3,>=1.23 in /usr/local/lib/python3.12/di
```

```
Requirement already satisfied: packaging>=20 in /usr/local/lib/python3.12/dis
```

```
Requirement already satisfied: pandas<4,>=1.4.0 in /usr/local/lib/python3.12/
```

```
Requirement already satisfied: pillow<13,>=7.1.0 in /usr/local/lib/python3.12
```

```
Collecting pydeck<1,>=0.8.0b4 (from streamlit)
```

```
  Downloading pydeck-0.9.2-py2.py3-none-any.whl.metadata (4.2 kB)
```

```
Requirement already satisfied: protobuf<8,>=3.20 in /usr/local/lib/python3.12
```

```
Requirement already satisfied: pyarrow>=7.0 in /usr/local/lib/python3.12/dist
```

```
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.12
```

```
Requirement already satisfied: tenacity<10,>=8.1.0 in /usr/local/lib/python3.
```

```
Requirement already satisfied: toml<2,>=0.10.1 in /usr/local/lib/python3.12/d
```

```
Requirement already satisfied: typing-extensions<5,>=4.10.0 in /usr/local/lib
```

```
Requirement already satisfied: starlette>=0.40.0 in /usr/local/lib/python3.12
```

```
Requirement already satisfied: uvicorn>=0.30.0 in /usr/local/lib/python3.12/d
```

```
Requirement already satisfied: httptools>=0.6.3 in /usr/local/lib/python3.12/
```

```
Requirement already satisfied: anyio>=4.0.0 in /usr/local/lib/python3.12/dist
```